

Verificação, Validação e Teste de Software (VVT) 2026.1

Week 4 - Conceitos básicos para a aplicação das atividades de VVT
Níveis de teste

Prof. Dr. Awdren de Lima Fontão



UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL



Níveis de teste

Os níveis de teste não surgem de forma isolada. Eles se conectam diretamente aos artefatos produzidos ao longo do desenvolvimento.

- **Requisitos do usuário:** orientam **teste de aceitação**
- **Requisitos de sistema/especificação funcional:** orientam **teste de sistema**
- **Arquitetura e projeto de alto nível:** orientam **teste de integração**
- **Projeto detalhado/implementação de módulos:** orientam **teste de unidade**

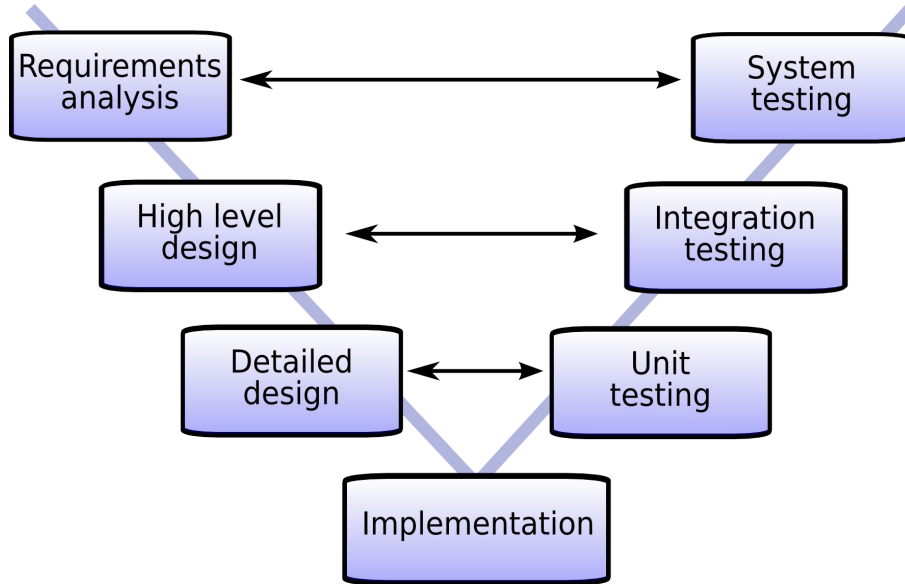
Requisito de usuário

O cliente deve conseguir finalizar um pedido pelo aplicativo.

Requisitos de sistema / especificação

- O sistema deve permitir selecionar ao menos um item antes da finalização.
- O sistema deve validar se o endereço de entrega está preenchido.

Modelo V



- **Teste de sistema** verifica se o sistema atende à especificação
- **Teste de integração** verifica se os componentes interagem corretamente
- **Teste de unidade** verifica o comportamento das menores partes do software

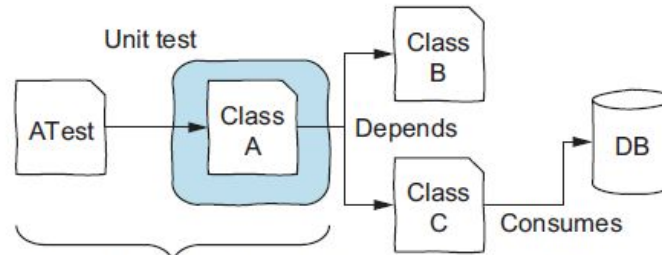
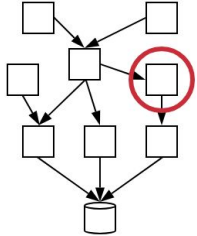
Nível	Foco principal	Exemplo
Unidade	Menor parte testável do software	método que calcula desconto
Integração	Comunicação entre módulos/serviços	pedido + pagamento
Sistema	Sistema completo em ambiente próximo do real	fluxo completo de compra
Aceitação	Necessidade do usuário/negócio	cliente consegue finalizar pedido

Exemplo guia: aplicativo de delivery

Fluxo principal do sistema:

1. Cliente faz login
2. Seleciona restaurante
3. Adiciona itens ao carrinho
4. Aplica cupom
5. Escolhe forma de pagamento
6. Finaliza pedido
7. Restaurante recebe o pedido
8. Cliente acompanha o status

Teste de Unidade



When unit testing class A, our focus is on testing A, as isolated as possible from the rest! If A depends on other classes, we have to decide whether to simulate them or to make our unit test a bit bigger.

Figure 1.5 Unit testing. Our goal is to test one unit of the system that is as isolated as possible from the rest of the system.

- É a fase do processo de teste em que se testam as menores unidades de software desenvolvidas. O universo alvo desse tipo de teste são os métodos dos objetos ou mesmo pequenos trechos de código
- Objetivo: encontrar falhas de funcionamento dentro de uma pequena parte do sistema funcionando independentemente do todo

Teste de unidade - vantagens

- **Testes unitários são rápidos:** normalmente levam apenas alguns milissegundos para serem executados. Isso permite testar grandes partes do sistema rapidamente. Uma suíte de testes automatizada e rápida oferece feedback constante, **aumentando a confiança** em mudanças evolutivas no software.
- **Testes unitários são fáceis de controlar:** testam um método fornecendo parâmetros específicos e comparando seu retorno com o resultado esperado. Esses valores de entrada e saída são fáceis de modificar e adaptar nos testes.
- **Testes unitários são fáceis de escrever:** não exigem configurações complexas. Como as unidades testadas são coesas e pequenas, o trabalho do testador se torna mais simples. Testes se tornam muito mais complicados quando envolvem bancos de dados, frontends e serviços web.

O principal benefício de testes de unidade é encontrar bugs, ainda na fase de desenvolvimento e antes que o código entre em produção, quando os custos de correção e os prejuízos podem ser maiores. Portanto, se um sistema de software tem bons testes, é mais difícil que os usuários finais sejam surpreendidos com bugs.

PROPRIEDADES "FIRST"

- **F**ast (Rápidos): devem executar em milissegundos para dar feedback rápido.
- **I**ndependent (Independentes): não devem depender da ordem de execução nem compartilhar estado global.
- **R**epeatable (Determinísticos): devem sempre produzir o mesmo resultado; evitar testes *flaky*.
- **S**elf-checking (Auto-verificáveis): o resultado deve ser automático, claro e binário: passou ou falhou.
- **T**imely (Escritos cedo): idealmente escritos o quanto antes, de preferência antes do código.

Teste de unidade - desvantagens

- **Testes unitários não refletem totalmente a realidade:** um sistema raramente é composto por uma única classe. A interação entre múltiplas classes pode fazer com que o sistema se comporte de forma diferente na execução real em comparação com os testes unitários.
- **Alguns bugs não são detectados:** certos erros só surgem na integração entre componentes, algo que os testes unitários puros não cobrem.
 - Exemplo: Em uma aplicação web complexa, testes individuais no backend e frontend podem passar, mas um bug pode aparecer somente quando ambos são integrados.
 - Outro caso: Em código multithread, cada unidade pode funcionar isoladamente, mas problemas surgem quando múltiplas threads interagem.

Exemplo de teste de unidade

Cenário: testar o método `calcularTaxaEntrega(distancia, horario, clima)`

O que está sendo testado?

- somente a lógica da função
- sem banco de dados
- sem interface gráfica
- sem API externa

Exemplo de dados:

- distância de 2 km → taxa R\$ 5,00
- distância de 10 km → taxa R\$ 12,00
- horário de pico → acréscimo previsto pela regra
- distância inválida → erro esperado

CT (ID: CT-UNIT-001)

Objetivo: Verificar se a taxa de entrega é calculada corretamente para distância acima de 5 km, em horário de pico e com chuva.

Entradas:

- distancia = 8
- horario = "pico"
- clima = "chuva"

Pré-condições: nenhuma

Passos:

1. Chamar `calcularTaxaEntrega(8, "pico", "chuva")`
2. Observar o valor retornado

Resultado esperado:

O sistema deve retornar **13.0**

Teste de integração

Unidades ou aplicações que foram testadas em separado são testadas de forma integrada. A interface entre as unidades integradas é testada.

Integração entre nosso código e partes externas a ele.

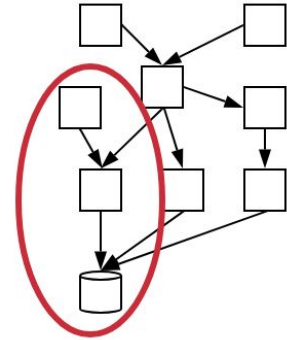
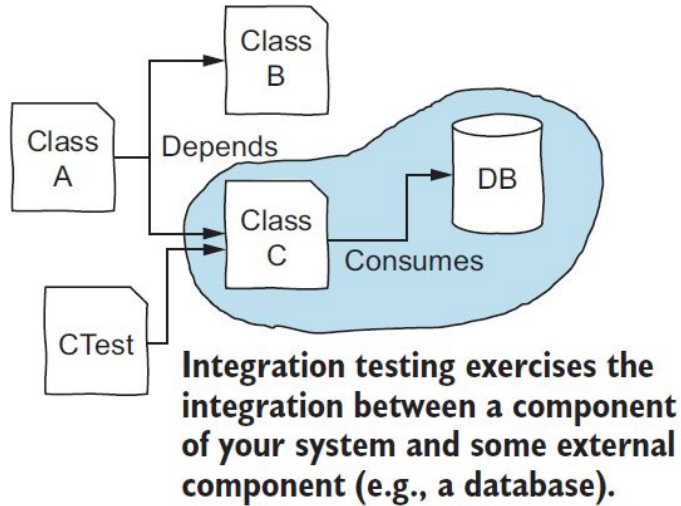


Figure 1.6 Integration testing. Our goal is to test whether our component integrates well with an external component.

Pontos de atenção com teste de integração

Testes de integração são mais complexos que testes unitários: enquanto testes unitários **isolam uma única unidade de código**, testes de integração precisam **percorrer o sistema inteiro** e lidar com componentes adicionais.

Testes com bancos de dados exigem esforço extra:

- Criar e configurar uma **instância isolada** do banco de dados;
- Atualizar o **esquema** conforme necessário;
- Definir o estado inicial do banco **inserindo ou removendo dados**;
- **Limpar tudo** após o teste para evitar interferências.

Outros testes de integração também demandam configuração: o mesmo esforço é necessário para **testes envolvendo serviços web, arquivos e operações de I/O**.

Exemplo de teste de integração

Cenário: integrar módulo de pedidos com gateway de pagamento

O que está sendo testado?

- envio correto dos dados do pagamento
- recebimento e interpretação da resposta
- tratamento de falhas da integração
- persistência do status do pagamento no sistema

Exemplos de validações:

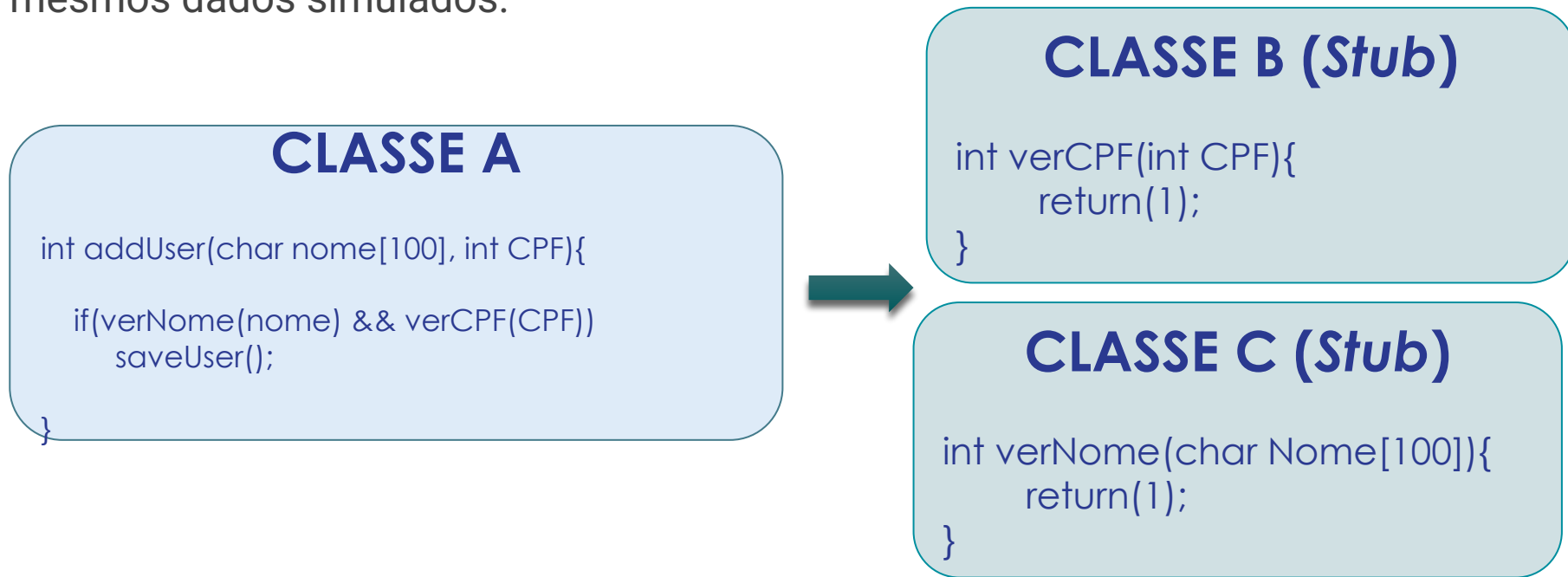
- pagamento aprovado → pedido marcado como confirmado
- pagamento recusado → pedido marcado como não pago
- timeout da API → sistema informa falha temporária
- resposta inconsistente → erro tratado adequadamente

Stubs e Drivers

- No contexto de teste de integração, usamos os elementos *stubs* e *drivers* -> **"mockar"**
- *Stubs*
 - pseudo-implementações de determinadas especificações (Casos básicos/triviais/esperados)
- *Drivers*
 - operações que automatizam testes de acordo com casos de teste

Teste de Integração - Stub (Dublê de Resposta)

É como um ator coadjuvante que finge ser um serviço real apenas para devolver respostas pré-programadas. Exemplo: Um sistema precisa consultar um banco de dados, mas ele ainda não está pronto. Criamos um stub que sempre retorna os mesmos dados simulados.



Um stub substitui um componente chamado pela unidade em teste e devolve respostas pré-definidas. DEVOLVE RESPOSTAS SIMULADAS.

Quando usar?

- quando a dependência real ainda não existe
- quando a dependência é complexa
- quando precisamos controlar exatamente a resposta

Exemplo no app de delivery:

O módulo de pedido precisa consultar o serviço de cupons.

Em vez de usar o serviço real, usamos um **stub** que sempre retorna:

- “cupom válido”
- desconto de 10%

```
def consultarTaxaCaptura(pokemon, pokebola):  
    return 0.8    # Stub: resposta fixa  
  
def capturarPokemon(pokemon, pokebola):  
    taxa = consultarTaxaCaptura(pokemon,  
pokebola)  
    if taxa >= 0.7:  
        return "Capturado"  
    return "Escapou"  
  
# Teste  
resultado = capturarPokemon("Pikachu", "Ultra  
Ball")  
print(resultado)    # Esperado: Capturado
```



Teste de Integração - Driver (Dublê de Chamador)

É como um piloto de testes, que simula uma parte do sistema que ainda não existe para chamar outro módulo.

Exemplo: Se temos uma biblioteca que processa pagamentos, mas o sistema que a chama ainda não foi feito, criamos um driver que simula esse chamador e envia os pedidos de pagamento.

CLASSE A

```
int verCPF(int CPF){  
    ...  
    return(1);  
    else  
    return(0);  
}
```



CLASSE B (Driver)

```
int testVerCPF(){  
    printf("%d", verCPF(1111111111)); // Válido  
    printf("%d", verCPF(0000000000)); // Inválido  
}
```

Um driver é um componente temporário usado para chamar e exercitar uma unidade que ainda não possui seu chamador real. CHAMA A UNIDADE EM TESTE

Quando usar?

- quando queremos testar um módulo isoladamente
- quando a interface ou módulo superior ainda não foi implementado

Exemplo no app de delivery:

Queremos testar o módulo `calcularFrete`, mas a tela de checkout ainda não existe.

Criamos um **driver** que envia CEP, distância e peso para esse módulo e exibe o resultado.

```
def calcularDano(ataque, tipo_atacante, tipo_defensor):  
    if ataque == "Ember" and tipo_defensor == "Grass":  
        return 40  
    return 20
```

Driver

```
def testarCalculoDano():  
    dano = calcularDano("Ember", "Fire", "Grass")  
    print(dano) # Esperado: 40
```

```
testarCalculoDano()
```



Teste de Integração

Estratégias mais citadas:

- Big-bang: construir tudo separadamente e integrar ao final. Pode dificultar a localização de defeitos;
- Bottom-up: módulos de nível mais baixo, ou seja, que não dependem um do outro. Módulo só é integrado quando os dependentes já foram integrados e testados. Não é necessário escrever *stubs*;
- Top-down: módulos de nível mais alto, verificando inicialmente os módulos com mais importância precisa de bons e vários *stubs*.

Teste de Sistema

Investiga o funcionamento da aplicação como um todo. Teste de execução dos fluxos dos casos de uso. Integração dos componentes de software com o ambiente operacional (similar ao de produção) é realizada e testada

Dado um input X, o sistema deve produzir o output Y corretamente.

When system testing, you want to exercise the entire system together, including all of its classes, dependencies, databases, web services, and whatever other components it may have.

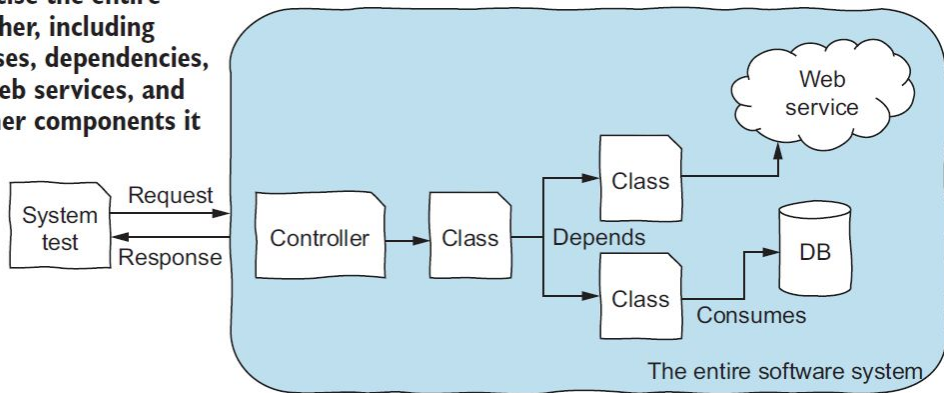


Figure 1.7 System testing. Our goal is to test the entire system and its components.

Teste de Sistema

- Exemplo de Problemas Detectados:
 - Não cumprimento a regras de negócio
 - Ex: cadastro de eleitores menores de 16 anos
 - Não atendimento a atributos de qualidade
 - Ex: tempo de resposta a uma requisição maior que o esperado



Teste de Sistema

- Tipos específicos de Teste de Sistema:
 - **Recuperação:** Força o software a falhar numa variedade de situações e verifica a capacidade de recuperação do produto
 - **Segurança:** Verifica se os mecanismos de proteção construídos para o sistema irão de fato protegê-lo de alguma utilização ou intrusão imprópria.
 - **Stress:** Executa o sistema de forma a exigir recursos em quantidade, frequência ou volume anormais
 - **Desempenho:** Avalia o desempenho do software quando integrado ao sistema. Normalmente está associado ao teste de stress

Pontos de atenção com o teste de sistema

- **Os testes possuem nível de realidade altíssimo:** Os usuários finais não executam métodos isolados: eles interagem com o sistema completo, como acessar uma página web, preencher um formulário e visualizar os resultados.
- **Testes de sistema são mais lentos:**
 - Precisam **inicializar e rodar todo o sistema**, incluindo todos os seus componentes.
 - Interagem com a aplicação real, podendo **demorar alguns segundos** para concluir.
 - Exemplo: Um teste que inicia **containers** para uma aplicação web e um banco de dados, faz uma requisição HTTP, busca dados no banco e retorna um JSON.

Pontos de atenção com o teste de sistema

Testes de sistema são mais difíceis de escrever:

- Requerem **configuração complexa**, como conectar, autenticar e garantir que o banco de dados contenha os dados corretos para o teste;
- Exigem **código adicional** para automação.

Testes de sistema são mais propensos a falhas intermitentes ("flaky tests"):

- Um **teste instável pode passar ou falhar** sem mudanças na configuração.
- Exemplo: Um teste que simula um clique em um botão pode falhar se o tempo de resposta do servidor variar levemente.
- **Incertezas** no ambiente de teste podem causar comportamentos inesperados.

Exemplo de teste de sistema

Cenário: validar o fluxo completo de realização de pedido no app

Fluxo avaliado:

- login do cliente
- seleção de itens
- cálculo do total
- aplicação de cupom
- escolha do pagamento
- finalização do pedido
- atualização do status

O que está sendo testado?

- comportamento do sistema como um todo
- regras de negócio funcionando em conjunto
- integração com os principais componentes
- resposta correta ao usuário

Sistema X E2E

Aspecto	Teste de Sistema	Teste E2E
Foco	comportamento do sistema completo	jornada ponta a ponta do usuário
Perspectiva	visão do sistema	visão do usuário e do negócio
Escopo	funcionalidades e atributos de qualidade	fluxo real completo entre componentes
Exemplo	cálculo final do pedido está correto	cliente compra e restaurante recebe o pedido

Exemplo de Sistema

Cenário: validar o fluxo completo de realização de pedido no app

Fluxo avaliado:

- login do cliente
- seleção de itens
- cálculo do total
- aplicação de cupom
- escolha do pagamento
- finalização do pedido
- atualização do status

O que está sendo testado?

- comportamento do sistema como um todo
- regras de negócio funcionando em conjunto
- integração com os principais componentes
- resposta correta ao usuário

Exemplo de E2E

Cenário: cliente realiza um pedido do início ao fim

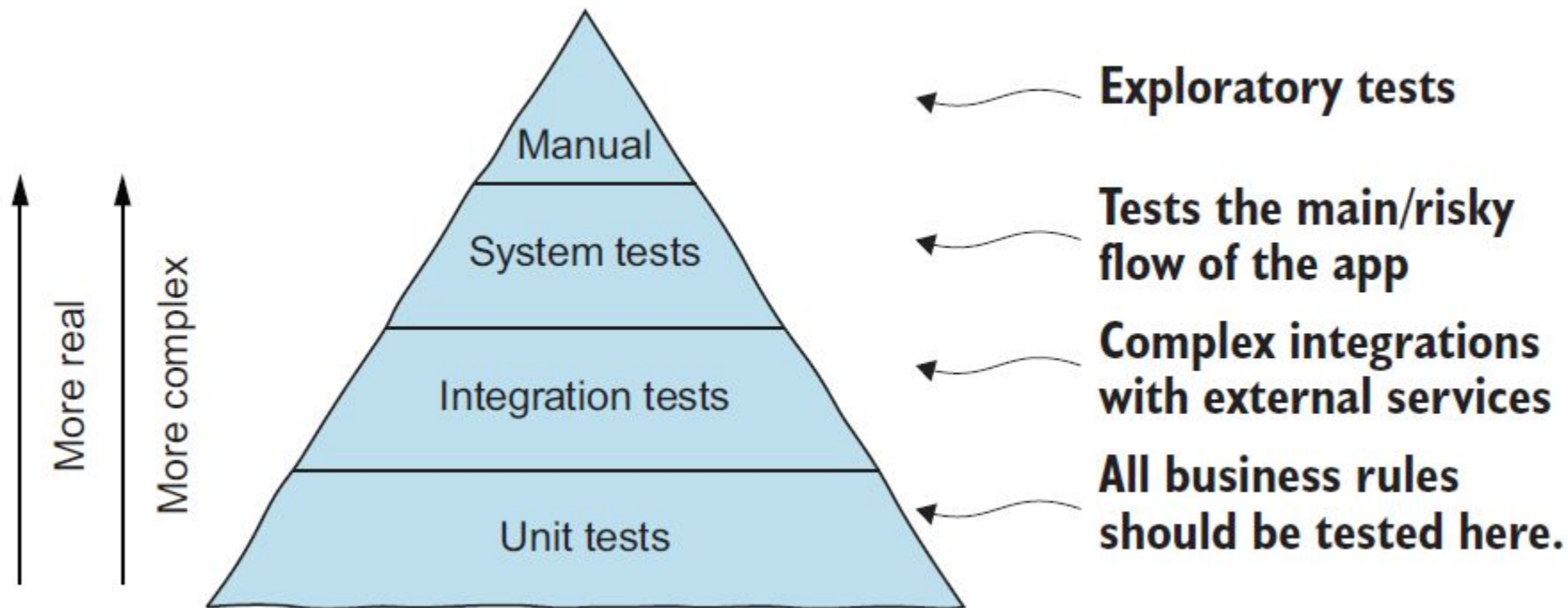
Fluxo ponta a ponta:

1. usuário faz login
2. escolhe restaurante
3. adiciona item ao carrinho
4. aplica cupom válido
5. paga com cartão
6. pedido é registrado
7. restaurante visualiza o pedido
8. cliente recebe confirmação

Validações possíveis:

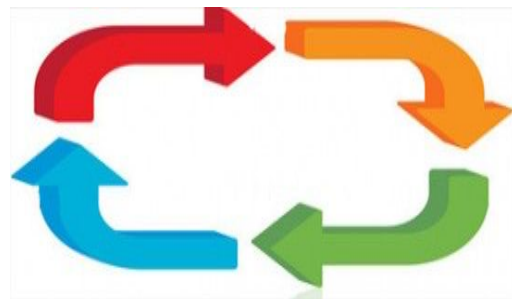
- cupom aplicado corretamente
- pagamento aprovado
- pedido persistido no banco
- restaurante notificado
- status exibido ao cliente

Quando usar cada nível?



Estratégias para Re-Teste

- Podem ocorrer para qualquer estratégia de teste, em caso de mudanças no software no planejamento dos testes
- Dois tipos:
 - Regressão
 - “Fumaça” (smoke testing)



Teste de Regressão

- Estratégia importante para redução de “efeitos colaterais”
- Consiste em se aplicar, a cada nova versão do software ou a cada ciclo, todos os testes que já foram aplicados nas versões ou ciclos de teste anteriores do sistema.
 - Execute testes de regressão toda vez que uma modificação maior é realizada no software (incluindo a integração de novos módulos)
 - Efetue a gerência de configuração

Para fechar (1):

Suzanne, uma testadora de software júnior, acabou de ingressar em uma **grande empresa de pagamentos online** nos Países Baixos. Como sua primeira tarefa, Suzanne **analisa os relatórios de bugs dos últimos dois anos**. Ela observa que **mais de 50% dos bugs ocorrem no módulo de pagamentos internacionais**.

Suzanne promete ao seu gerente que **criará casos de teste que cubram completamente o módulo de pagamentos internacionais** e, assim, **encontrará todos os bugs**.

Qual dos seguintes **princípios de teste** pode explicar por que isso não é possível?

- A) **Paradoxo do pesticida**
- B) **Teste exaustivo**
- C) **Testar cedo**
- D) **Agrupamento de defeitos**

Para fechar (2):

John acredita fortemente em **testes unitários**. Na verdade, **esse é o único tipo de teste que ele realiza** em qualquer projeto em que participa.

Qual dos seguintes **princípios de teste não** ajudará a convencer John de que ele deve **abandonar sua abordagem de “apenas testes unitários”**?

- A) **Paradoxo do pesticida**
- B) **Testes são dependentes do contexto**
- C) **Falácia da ausência de erros**
- D) **Testar cedo**